

OOP and Exception Handling in PHP

1. Creating Classes and Objects

What is a Class?

A **Class** is a blueprint or template for creating objects.

Example

Suppose you want to create many students.

Every student has:

- Name
- Roll Number
- Course

Instead of writing the same code repeatedly, create a class.

```
<?php

class Student
{
    public $name;
    public $rollno;
    public $course;

    public function display()
    {
        echo "Name: ".$this->name."<br>";
        echo "Roll No: ".$this->rollno."<br>";
        echo "Course: ".$this->course."<br>";
    }
}

?>
```

Creating Objects

```
<?php

$s1 = new Student();

$s1->name = "Rahul";
$s1->rollno = 101;
$s1->course = "BCA";

$s1->display();

?>
```

Output

```
Name: Rahul
Roll No: 101
Course: BCA
```

Real-Life Example

Class = Car Blueprint

Object 1

- Color = Red
- Brand = BMW

Object 2

- Color = Blue
- Brand = Audi

One blueprint can create many cars.

2. Using Constructors

What is a Constructor?

A **Constructor** is a special function that automatically runs whenever an object is created.

Constructor name:

```
__construct()
```

Example

```
<?php

class Student
{
    public $name;

    function __construct($n)
    {
        $this->name = $n;
    }

    function display()
    {
        echo "Welcome ".$this->name;
    }
}

$s1 = new Student("Amit");

$s1->display();

?>
```

Output

Welcome Amit

Why use Constructors?

Without constructor:

Create object

↓

Assign values manually

↓

Use object

With constructor:

Create object

↓

Values assigned automatically

Less work and cleaner code.

3. Property Visibility

Visibility means **who can access the data**.

There are **3 types**.

Visibility	Access
public	Everyone
private	Only inside the class
protected	Class and child classes

Public

Anyone can access.

```
<?php
class Student
{
    public $name = "Riya";
}

$s = new Student();

echo $s->name;

?>
```

Output

Riya

Private

Cannot access outside the class.

```
<?php
class Student
{
    private $marks = 90;

    function show()
    {
        echo $this->marks;
    }
}

$s = new Student();

$s->show();

?>
```

Output

90

This will cause an error:

```
echo $s->marks;
```

Protected

Accessible only inside the class and inherited classes.

```
protected $salary;
```

Used mostly with inheritance.

4. Inheritance

Inheritance means **one class acquires the properties and methods of another class**.

Think of it like a child inheriting qualities from parents.

Example

Parent Class

```
<?php
class Animal
```

```

{
    public function sound()
    {
        echo "Animals make sounds.<br>";
    }
}

class Dog extends Animal
{
    public function bark()
    {
        echo "Dog barks.";
    }
}

$d = new Dog();

$d->sound();
$d->bark();

?>

```

Output

```

Animals make sounds.
Dog barks.

```

Benefits

- Reuse code
- Less programming
- Easy maintenance

5. Method Overriding

Method overriding means **the child class provides its own version of a method already defined in the parent class.**

Example

```

<?php

class Animal
{
    public function sound()
    {
        echo "Animal Sound";
    }
}

class Dog extends Animal
{
    public function sound()
    {

```

```
        echo "Dog Barks";
    }
}

$d = new Dog();

$d->sound();

?>
```

Output

Dog Barks

The child method replaces the parent's method.

Real-Life Example

Parent:

Teacher teaches.

Child:

Math Teacher teaches Mathematics.

The child gives a more specific behavior.

6. Exception Handling

Sometimes programs encounter errors.

Instead of crashing, we can **handle errors gracefully** using **Exception Handling**.

Keywords:

- try
- catch
- finally
- throw

try Block

Put code that may produce an error.

```
try
{
    // risky code
}
```

throw

Used to create an exception.

```
throw new Exception("Invalid Age");
```

catch

Catches the exception.

```
catch(Exception $e)
{
    echo $e->getMessage();
}
```

finally

Runs whether an exception occurs or not.

Used for cleanup tasks like closing files or database connections.

Complete Example

```
<?php

$age = 15;

try
{
    if($age < 18)
    {
        throw new Exception("You are not eligible.");
    }

    echo "Welcome";
}

catch(Exception $e)
{
    echo "Error: ".$e->getMessage();
}

finally
{
    echo "<br>Program Finished.";
}

?>
```

Output

```
Error: You are not eligible.
Program Finished.
```

Flow Diagram

```
Start
↓
try
↓
Any Error?
↓
Yes -----> catch
↓
No
↓
finally
↓
End
```

Real-Life Example

Suppose you are withdrawing money from an ATM.

```
Insert Card
↓
Enter PIN
↓
Correct?
↓
Yes → Money
↓
No → Error Message
↓
ATM Returns Card
```

Returning the card is like the **finally** block—it always happens.

7. Input Validation using Regular Expressions (Regex)

What is Validation?

Validation checks whether user input follows the expected format.

Examples:

- ✓ Email
- ✓ Mobile Number
- ✓ Password
- ✓ PIN Code

What is Regex?

Regex (Regular Expression) is a pattern used to match text.

PHP function:

```
preg_match()
```

Syntax

```
preg_match(pattern, string)
```

Returns:

- **1** → Match found
- **0** → No match

Example 1: Validate Name

Only alphabets allowed.

```
<?php
$name = "Rahul";

if(preg_match("/^[A-Za-z]+$/", $name))
{
    echo "Valid Name";
}
else
```

```
{
    echo "Invalid Name";
}

?>
```

Output

Valid Name

Example 2: Validate Mobile Number

Exactly 10 digits.

```
<?php

$mobile = "9876543210";

if(preg_match("/^[0-9]{10}$/", $mobile))
{
    echo "Valid Mobile";
}
else
{
    echo "Invalid Mobile";
}

?>
```

Output

Valid Mobile

Example 3: Validate Email

```
<?php

$email = "student@gmail.com";

if(filter_var($email, FILTER_VALIDATE_EMAIL))
{
    echo "Valid Email";
}
else
{
    echo "Invalid Email";
}

?>
```

Output

Valid Email

Note: While email validation can be done with regex, PHP's built-in `filter_var()` is generally recommended because it is simpler and follows standard email validation rules.

Common Regex Symbols

Symbol	Meaning	Example
<code>^</code>	Start of string	<code>^A</code>
<code>\$</code>	End of string	<code>Z\$</code>
<code>.</code>	Any single character	<code>a.c</code>
<code>*</code>	Zero or more occurrences	<code>ab*</code>
<code>+</code>	One or more occurrences	<code>ab+</code>
<code>?</code>	Zero or one occurrence	<code>colou?r</code>
<code>{n}</code>	Exactly n times	<code>[0-9]{10}</code>
<code>{n,m}</code>	Between n and m times	<code>[A-Z]{2,5}</code>
<code>[]</code>	Character set	<code>[A-Za-z]</code>
<code>[^]</code>	Not in the set	<code>[^0-9]</code>

Advantages of OOP

- Code reuse through inheritance.
- Better organization with classes and objects.
- Easier maintenance and debugging.
- Improved security using property visibility.
- Easier to extend applications.

Advantages of Exception Handling

- Prevents program crashes.
- Makes debugging easier.
- Improves user experience with meaningful error messages.
- Separates error-handling logic from normal program flow.
- Ensures cleanup using the `finally` block.

Quick Revision

Topic	Key Idea
Class	Blueprint for objects
Object	Instance of a class
Constructor	Runs automatically when an object is created
Public	Accessible from anywhere
Private	Accessible only within the class
Protected	Accessible within the class and subclasses
Inheritance	Child class reuses parent class features
Method Overriding	Child class replaces a parent method
<code>try</code>	Contains code that may throw an exception
<code>throw</code>	Creates an exception
<code>catch</code>	Handles an exception
<code>finally</code>	Executes whether or not an exception occurs
Regex	Pattern used to validate text
<code>preg_match()</code>	Matches input against a regex pattern
<code>filter_var()</code>	Recommended for validating email addresses